

Compliance & Secure Coding Practices

in 25-26J-043

Project ID: 25-26J-043

Project Title: CAPSLock - A Kubernetes-Native Multi-Environment Deployment System with Integrated ICAP Security Orchestration

Student Name: Kulatunga K K M D, Raigambandarage N K, Guruge S A L, PANDUKABHAYA H D T

Student ID: IT22353634, IT22338716, IT22353634, IT22345028

Supervisor: Mr.Amila Nuwan Senarathne, Mr.Kavinga Abeywardena

Date of Submission: 30/04/2026

1. Introduction

CAPSLock is a Kubernetes-native research platform that addresses the critical gap of enforcing consistent security policies across multi-environment Kubernetes deployments. The system consists of four tightly integrated components: MEDS (Multi-Environment Deployment System), EAPE (Environment-Aware Policy Engine), SSDLB (Smart Service Discovery Load Balancer), and an ICAP Operator - a custom Kubernetes controller. The project's primary objective is to embed ICAP-based content scanning, risk-aware validation, and policy lifecycle management directly into the deployment pipeline, replacing ad-hoc security toolchains with a unified, automated framework.

Compliance and secure coding are foundational to this system - not afterthoughts - because the system itself enforces CIS Kubernetes Benchmark v1.9 and PCI-DSS v4.0 controls programmatically. Every component was built with secure input validation, structured logging without data leakage, mTLS inter-service communication, RBAC least-privilege principles, and cryptographically verifiable audit trails.

2. Secure Coding Standards

OWASP Secure Coding Practices - Applied across all three Python FastAPI services (MEDS port 8000, EAPE Python bridge port 8001, SSDLB port 8082). Input validation is enforced via Pydantic schema models on every API request body and path parameter. CORS is explicitly configured. Error responses never expose internal stack traces or system paths. API endpoints follow least-privilege response design - returning only the fields the caller requires.

CERT Coding Standards (Go) - Applied to the Go components: the EAPE policy engine (pkg/ packages) and the ICAP Operator (controller-runtime). Go's static type system and explicit error-return patterns are used consistently. Mutex-protected shared state (sync.RWMutex in the PolicyEngine struct) prevents race conditions. All external inputs are validated before use. No panic() calls exist in production paths.

ISO/IEC 27001 - The immutable audit log with SHA-256 hash-chaining implements the non-repudiation control requirement. Policy versioning with rollback implements change management controls. RBAC manifests (Kubernetes ServiceAccount, Role, RoleBinding) follow the documented least-privilege access control principle. TLS/mTLS is configurable via environment variables (CA_CERT_PATH, MTLS_CERT_PATH, MTLS_KEY_PATH) for all inter-component HTTP communication.

CIS Kubernetes Benchmark v1.9 - Directly implemented as automated checks in the EAPE policy engine (pkg/compliance/cis/ - sections 4.1 RBAC, 4.2 Pods, 4.3 Network, 4.4 Secrets, 4.5 Namespaces). The system enforces these as admission controls via generated OPA Rego and Kyverno ClusterPolicy manifests.

PCI-DSS v4.0 - Validated programmatically in pkg/compliance/pcidss/. SSDLB maps every routing decision to specific PCI-DSS control references (10.2.1, 10.3.1, 10.6.1, 10.2.2) and logs them in structured JSON.

3. Key Secure Coding Techniques Implemented with Examples

- **Input Validation (Pydantic)**: All API inputs are declared as typed Pydantic models (components/meds-research/meds/models/promotion.py). FastAPI automatically rejects non-conforming requests. Path parameters are also validated - e.g., version in SSDLB is checked against an allowed set {"a", "b", "c", "spread"}. Example: ApplicationRef and PromotionSpec Pydantic models enforce typed fields; SSDLB validates version length (≤32 chars) and alphanumeric format.

- **Immutable Audit Logging (SHA-256 Hash Chain)**: Every audit event (meds/audit/log.py) computes a SHA-256 digest over its own fields and includes the previous event's hash, forming a verifiable chain. Tampering with any event breaks the chain. Integrity is verified via /api/audit/verify.

- **Authentication & Authorization - RBAC Least Privilege**: The ICAP Operator's RBAC annotations (config/rbac/role.yaml) define exactly which Kubernetes resources it can access. No cluster-admin, no wildcard verbs. Each component has its own ServiceAccount with minimum required permissions. CIS Check 4.1.1 enforces cluster-admin binding detection.

- **Encryption - mTLS for Inter-Service Communication**: All inter-component HTTP calls support mutual TLS through environment-variable-injected certificate paths (CA_CERT_PATH, MTLS_CERT_PATH, MTLS_KEY_PATH). In Kubernetes, Istio service mesh enforces mTLS automatically between all CAPSLock pods via the sidecar.istio.io/inject annotation on the ICAP Operator pod template.

- **Error Handling - Structured Logging Without Data Leakage**: Structured JSON logging via structlog (meds/utils/logger.py) produces machine-parseable events with no raw exception strings exposed to end users. API error responses use FastAPI's HTTPException with controlled messages only - no stack traces, no internal paths.

- **ICAP Content Scanning - RFC 3507 Protocol Implementation**: Every deployment artifact is scanned via the real ICAP protocol over TCP port 1344 before promotion is allowed (meds/icap/scanner.py). The scanner sends an RFC 3507 RESPMOD request to the ClamAV-backed c-ICAP server. A 204 response clears the artifact; a 200 with X-Infection-Found immediately rejects the promotion.

- **Risk-Based Input Control - Promotion State Machine:** Promotions follow a one-way state machine - no backward or stage-skipping transitions are permitted. A 6-factor weighted risk score gates every promotion (promotion_controller.py). Score exceeding the environment threshold (Dev=80, Staging=60, Prod=40) causes automatic rejection.

- **Kubernetes CRD Validation - Declarative Input Constraints:** CRD fields use kubebuilder markers for server-side validation (api/v1alpha1/icapservice_types.go), preventing misconfiguration at the Kubernetes API layer. Example: Replicas validated 1–10, TargetHealthScore validated 50–100.

4. Tools & Technologies Used for Secure Coding

- **Static Analysis & Linting:** golangci-lint for Go static analysis (ICAP Operator and EAPE policy engine); Python type checking via Pydantic (runtime) and built-in type hints across all FastAPI services; Kubebuilder marker validation for declarative CRD field constraints.

- **Security Scanning:** ClamAV antivirus engine embedded in every ICAP pod via clamav/clamav:latest sidecar container with daily signature-update CronJob; EICAR test vectors for pipeline verification; OPA (Open Policy Agent) generated Rego policies for admission control; Kyverno generated ClusterPolicy manifests for pod security standards.

- **Monitoring & Compliance Verification:** Prometheus scrapes metrics from all components; Grafana dashboard for real-time health score visualization; alerting rules fire when ICAP health score drops below thresholds.

- **Testing:** pytest full test suite covering the promotion controller, risk scorer, USLO engine, ICAP scanner, and end-to-end pipeline; Locust load and benchmark testing for SSDLB; controller-runtime envtest for Go integration tests of the ICAP Operator CRD reconciliation loop.

5. Compliance Standards

PCI-DSS v4.0 - CAPSLock implements PCI-DSS v4.0 compliance programmatically in two ways. First, the EAPE policy engine's pkg/compliance/pcidss/ module runs automated PCI-DSS requirement checks against Kubernetes namespace configurations and produces pass/fail reports with remediation guidance. Second, SSDLB maps every routing decision event to specific PCI-DSS control references and records them in structured logs - for example, predictive_spread_entered maps to controls 10.2.1 and 10.3.1 (audit logs for all access), and icap_health_forced_spread maps to 10.6.1 (security monitoring). The production policy template (prod-policy.yaml) enforces PCI-DSS requirements: restricted pod security standards, required network policies, external secrets manager, and no secrets as environment variables.

CIS Kubernetes Benchmark v1.9 - The EAPE policy engine implements all CIS Benchmark sections 4.1–4.5 as automated validators in Go. Section 4.1 (8 checks): RBAC and Service Accounts - cluster-admin restrictions, secrets access minimization, wildcard permission detection. Section 4.2: Pod Security - privileged container detection, host path restrictions, run-as-non-root enforcement. Section 4.3: Network Policies - network policy existence and default-deny enforcement. Section 4.4: Secrets -

environment variable secrets detection, external secrets manager enforcement. Section 4.5: Namespace-level controls. Each check produces a RuleViolation object with RuleID, Severity, Remediation, and References to the official benchmark. Compliance scores feed into the MEDS risk scorer as the compliance_posture factor (weight: 10%), directly affecting promotion decisions.

Data Protection & Audit Compliance - All deployment decisions and policy changes are recorded in the immutable SHA-256 hash-chained audit log. The chain integrity is verifiable via the /api/audit/verify endpoint. No personal data is processed by CAPSLock - it operates on Kubernetes deployment metadata only (namespace names, version strings, policy names). The audit log records actor identity, timestamps, environment, and event type for every promotion, approval, rejection, and rollback, satisfying non-repudiation requirements under ISO/IEC 27001.

6. Challenges & Mitigation Strategies

Challenge 1: ICAP Protocol Incompatibility with Kubernetes Ephemerality - Traditional ICAP implementations (RFC 3507) rely on persistent proxy connections. Kubernetes pods start and stop within seconds, making persistent connections unreliable. The mitigation was a three-layer fallback scanner (meds/icap/scanner.py): Layer 1 attempts a real RFC 3507 RESPMOD request over TCP to the ICAP Operator when ICAP_SERVICE_HOST is set; Layer 2 falls back to the policy-engine compliance gate when running in docker-compose; Layer 3 uses deterministic simulation seeded by hashlib.sha256 for offline and CI environments.

Challenge 2: Race Conditions in Shared Routing State - The SSDLB controller maintains in-memory routing state (mode, last_selected, timestamps) that is read and written by concurrent HTTP requests. The mitigation was a file-based atomic state store using Python's os.replace() for atomic writes - write to a temp file, then rename - combined with guardrail timers (60-second cooldown, minimum metric window) that prevent rapid state oscillation. The Go policy engine uses sync.RWMutex protecting the appliedPolicies map for concurrent namespace policy reads and writes.

Challenge 3: Policy Conflicts Between Environments - Different environments legitimately require different policies, but naive policy evolution causes conflicts - for example, PCI-DSS requires pod_security_standard: restricted, but development environments use baseline. The mitigation was the USLO (Unified Security Lifecycle Orchestration) Engine, which formalises the lifecycle: policies enter as Audit mode (violations logged, not blocked) in staging, then graduate to Enforce mode in production. The EAPE conflict resolver uses a compliance-coverage scoring algorithm to deterministically select winners when two policies conflict.

8. References

- OWASP Secure Coding Practices: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>
- CERT Secure Coding Standards: <https://www.cert.org/secure-coding/>

- ISO/IEC 27001 Information Security Management: <https://www.iso.org/isoiec-27001-information-security.html>
- CIS Kubernetes Benchmark v1.9: <https://www.cisecurity.org/benchmark/kubernetes>
- PCI-DSS v4.0: https://www.pcisecuritystandards.org/document_library/
- OPA Gatekeeper: <https://open-policy-agent.github.io/gatekeeper/>
- Kyverno Policy Engine: <https://kyverno.io/docs/>
- Kubernetes controller-runtime: <https://github.com/kubernetes-sigs/controller-runtime>
- RFC 3507 - ICAP Protocol: <https://datatracker.ietf.org/doc/html/rfc3507>
- Istio Service Mesh mTLS: <https://istio.io/latest/docs/concepts/security/>
- golangci-lint: <https://golangci-lint.run/>
- ClamAV Antivirus Engine: <https://www.clamav.net/>